

How the change points are found

Yuli Tshuva · September 2026

The goal. Cut a sequence into a few pieces, so that each piece is a simple, single trend. The cut positions are the change points.

Step 1: score one piece. Take any stretch of the sequence from a to b . Draw the best straight line through it and measure how much the sequence misses that line:

$$c(a, b) = \min_{\alpha, \gamma} \sum_{t=a}^{b-1} (y_t - \alpha - \gamma t)^2.$$

A small score means the stretch really is one clean trend. A large score means something happens inside it that a single line cannot follow.

Step 2: find the best cuts, for a fixed number of cuts. We want the set of cuts whose pieces have the smallest total score. Trying every possible set is hopeless — there are astronomically many. But the problem has a useful shape: *the best way to cut the first t points is the best way to cut the first s points, plus one final piece from s to t .* So if we already know the answer for every shorter prefix, we get the answer for t by trying each possible last cut s and keeping the winner:

$$F(t) = \min_{s < t} [F(s) + c(s, t)], \quad F(0) = 0.$$

Sweeping t from left to right fills the whole table, and following the winners backwards recovers the cuts. This is dynamic programming. It examines every segmentation implicitly, so the result is genuinely the best one — not a good guess. Adding a counter k for the number of cuts used gives the best segmentation for each number of cuts at the same time.

Step 3: choose how many cuts. More cuts always fit better, so we need a stopping rule. Let c_K be the total leftover error using K cuts, and c_0 the error with no cuts at all — one line through everything. We take the *fewest* cuts that bring the error down far enough:

$$K^* = \min \{ K : c_K / c_0 \leq \text{tol} \}, \quad \text{tol} = 0.009.$$

In plain words: keep adding cuts until the pieces explain all but 0.9% of what a single straight line failed to explain, then stop. One number, tol , controls the whole thing, and it means the same thing on every sequence because it is a fraction, not an absolute size.

Why it is fast. Each score $c(a, b)$ looks like it needs a fresh line fit, but a line fit only depends on five running sums of y , y^2 , ty , t and t^2 . Storing those sums once lets any $c(a, b)$ be read off in a single step, so the whole search takes about a tenth of a second for a sequence of 1000 points.

What replaced what. The previous detector made a chain of local decisions — threshold the slope, filter by prominence, merge nearby points, drop points facing the same way — with six thresholds and no way to know if the answer was good. All of that is gone. There is one thing being minimized, one number to set, and the answer is optimal by construction.